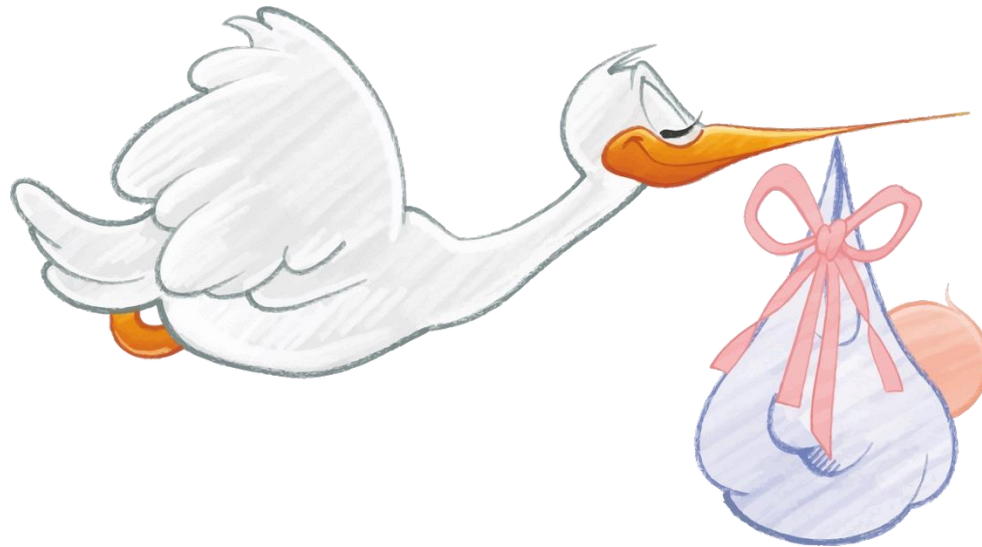


Dependency Injection



Wo kommen eigentlich die Objekte her?

new Baby();



new: wo und wann?

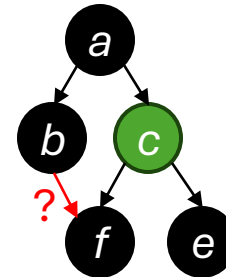
Grundproblem: Wo und wann sollen Objekte in einer Software erzeugt werden?

🤔 **Idee 1:** Immer in der Klasse, wo sie benötigt werden.

🤔 **Problem 1:**

c erzeugt f und e .

Wenn b auch auf dasselbe f zugreifen soll,
ist doof, weil b nicht einfach an f herankommt.



```
class C {  
    private F f = new F();  
    private E e = new E();  
  
    // do something with f and e...  
}
```

```
class B {  
    // How to get object f in B??  
    private F f;  
  
    // do something with f...  
}
```

🤔 **Problem 2:**

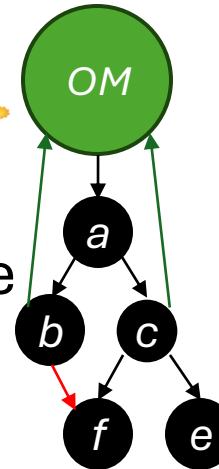
Wenn für c ein Unit Test geschrieben wird, müssen f und e mitgetestet werden,
da diese fest verdrahtete Abhängigkeiten von c sind.

new: wo und wann?

Grundproblem: Wo und wann sollen Objekte in einer Software erzeugt werden?



Idee 2: Zentraler "Objekt-Manager" (OM)



OM erzeugt *a*, *b*, *c*, *f* und *e*.

Zugriff auf die Objekte erfolgt über statische Getter in OM.

```
class OM {  
    private static A a = new A();  
    // ...  
    private static E e = new E();  
    public static getA() {return a;}  
    // ...  
    public static getE() {return e;}  
}
```



Probleme:

Die Objekte von OM lassen sich für Unit-Tests nicht leicht austauschen (erfordert z. B. Hilfswerkzeuge wie Mockito).

Man könnte statische Setter in OM hinzufügen, aber das würde zu ungewollten Seiteneffekten führen, da die Ausführungsreihenfolge der Setter dann wichtig wäre.

```
class C {  
    private F f = OM.getF();  
    // do something with f...  
}
```

```
class B {  
    private F f = OM.getF();  
    // do something with f...  
}
```

new: wo und wann?

Grundproblem: Wo und wann sollen Objekte in einer Software erzeugt werden?

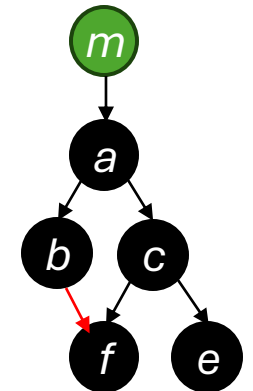
🤔 **Idee 3:** Ganz "oben" in der main-Methode (*m*).

m erstellt *a*, *b*, *c*, *f* und *e*.

m übergibt *b* und *c* an *a* sowie *f* und *e* an *c* (per Konstruktor).

Falls *b* auch *b* benötigt, kann *m* einfach *f* an *b* übergeben.

Für Unit Tests lassen sich Klassen über den Konstruktor austauschen! 😊!



😞 **Problem:**

Bei großen Projekten mit vielen Klassen und unterschiedlichen Konstruktoren wird das schnell kompliziert wegen der Erzeugungsreihenfolge.

```
public static void main(String[] args) {  
    F f = new F();  
    E e = new E();  
    C c = new C(f, e);  
    B b = new B();  
    // new B(f);  
    A a = new A(b, c);  
}
```

Dependency Manager

Kann man die lästige Verwaltung von Objekten nicht delegieren?

Ja, es gibt Dependency Manager Frameworks, die genau dies machen. Ein bekanntes Beispiel für Java ist z. B. [Spring](#).



Dependency Injection

Abhängigkeiten über einen Konstruktor (oder Setter) zu übergeben, nennt man **Dependency Injection**.

```
interface TranslationService {  
    String translate(String text);  
}
```

```
class Translator {
```

```
    private TranslationService translationService;
```

```
    Translator(TranslationService translationService) {  
        this.translationService = translationService; // speichere das übergebene Objekt für später im Attribut ab.  
    }
```

```
    String translate(String text) {  
        return translationService.translate(text);  
    }
```

```
    public static void main(String[] args) {
```

```
        var deEnTranslator = new Translator(new GermanEnglishTranslator()); // Erzeuge alle Objekte und erfülle ihre Abhängigkeiten mittels Dependency Injection.  
        System.out.println(deEnTranslator.translate("Hallo")); // returns Hello
```

```
        var deKlTranslator = new Translator(new GermanKlingonTranslator()); // Erzeuge alle Objekte und erfülle ihre Abhängigkeiten mittels Dependency Injection.  
        System.out.println(deKlTranslator.translate("Hallo")); // returns Kapla
```

```
    }  
}
```

"Die Klasse Translator hängt von der Klasse TranslationService ab."
Das benötigte Objekt (die Abhängigkeit) wird hier über den Konstruktor injiziert z. B. von der main-Methode oder von einem Testfall (Unit Test).

Hier wird die injizierte Abhängigkeit vom Translator aufgerufen.

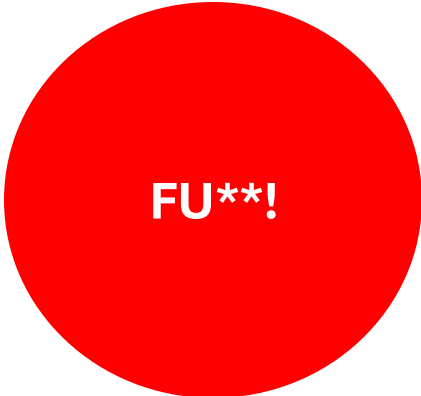
Unit Test: Recap

Ein **Unit-Test** ist ein Test, der sich auf eine einzelne Aufgabe einer **einzelnen Klasse** bezieht, die oft als *System Under Test* (SUT) bezeichnet wird.

Der Zweck von Unit-Tests ist es, zu überprüfen, ob der Code in einem SUT funktioniert.



Alles ist in
Ordnung.
Schlaf gut.



FU**!

Unit Test

brewCoffee()

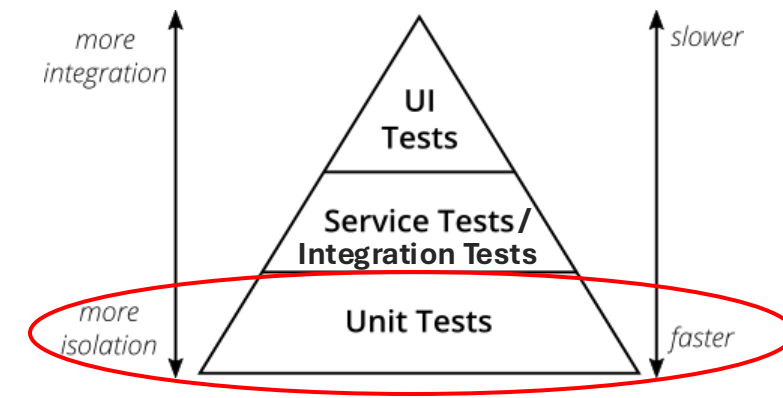


Integrations
test

Testdoubles



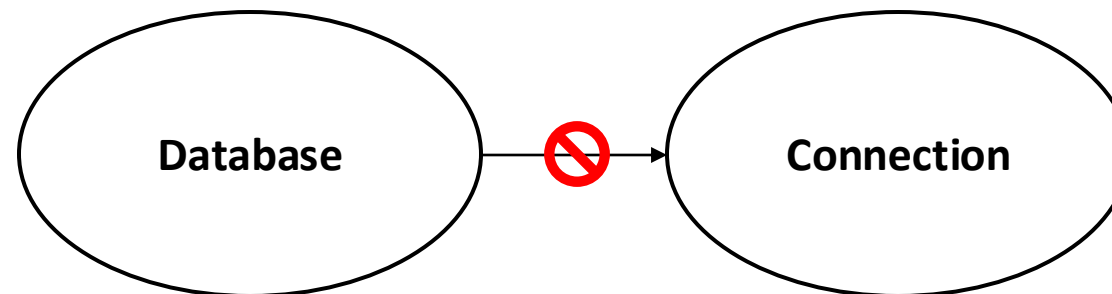
Isolierte Unit Tests



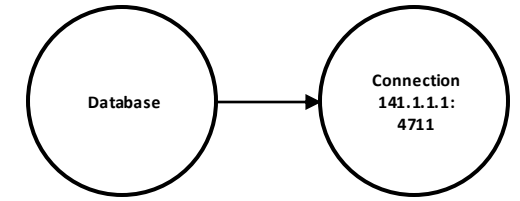
Beispiel:

Es gibt eine Klasse **Database**, die eine Abhängigkeit zur Klasse **Connection** hat ("Database verwendet Connection").

Im **Unit Test** soll die Klasse **Database** **isoliert getestet** werden. Um das zu erreichen, muss während des Unit Tests die Klasse **Connection** von der Klasse **Database** **entkoppelt** werden.



Beispiel: Hardcodiertes Objekt



```
class Database {  
  
    private final Connection connection = new Connection("141.1.1.1", 4711); // hardcodierte Zuweisung  
                                         // nicht änderbar  
  
    public Person findPersonById(int personId) {  
        return connection.query(personId);  
    }  
}
```

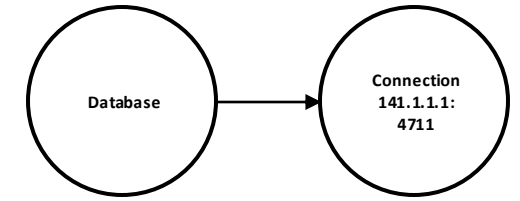
Problem:

Das Connection-Attribut erhält ein hardcodiertes Objekt zugewiesen, das sich während eines Unit Tests nicht ändern lässt.

Wenn wir einen Unit Test für die Klasse Database schreiben wollen, wollen wir nicht die **reale Datenbank** des Produktsystems verwenden sondern ein **Testdouble**!



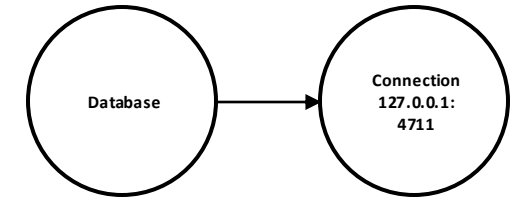
Dependency Injection 1/2



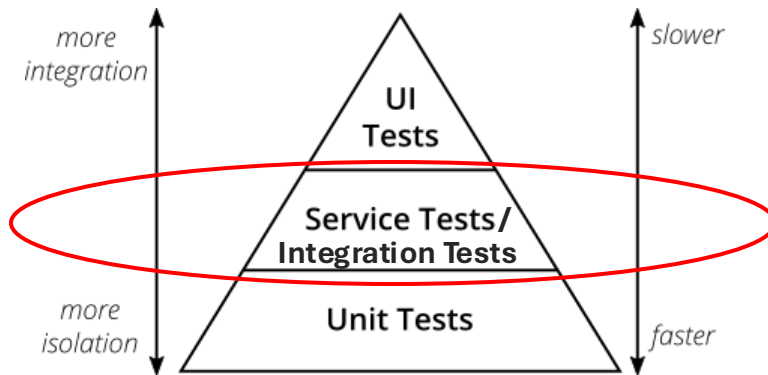
```
class Database {  
    private final Connection connection;  
  
    public Database(Connection connection) {  
        this.connection = connection; // connection wird nicht länger hardcodiert zugewiesen sondern dynamisch (aka Dependency Injection).  
    }  
  
    public Person findPersonById(int personId) {  
        return connection.query(personId); // Database verwendet die connection (Database ist abhängig von Connection).  
    }  
  
    // Beispiel: App für Produktion initialisieren und starten:  
    public static void main(String[] args) {  
        String host = "141.1.1.1"; // Verwende IP-Adresse der Produktions-Datenbank.  
        int port = 4711;  
        Connection connection = new Connection(host, port); // Erzeuge Dependency für Produktions-Database.  
        Database database = new Database(connection); // Dependency Injection.  
        database.findPersonById(1); // Database verwendet intern Produktions-Database für die Abfragen.  
    }  
}
```

Dependency Injection (DI) bedeutet, dass eine **Abhängigkeit (ein Objekt)** über einen **Konstruktor oder Methode (Setter)** übergeben wird. Dadurch lässt sich das **Objekt austauschen**.

Dependency Injection 2/2 (Integrationstest)



```
class Database {  
    private final Connection connection;  
  
    public Database(Connection connection) {  
        this.connection = connection;  
    }  
  
    public Person findPersonById(int personId) {  
        return connection.query(personId);  
    }  
}
```



Integrationstest, der eine reale Testdatenbank verwendet:

```
class DatabaseTest {  
  
    private static Database database;  
  
    @BeforeAll  
    static void beforeAll() {  
        String host = "127.0.0.1"; // Verwende Adresse einer Testdatenbank.  
        int port = 4711;  
        Connection testDatabaseConnection = new Connection(host, port);  
        database = new Database(testDatabaseConnection); // DI  
    }  
  
    @Test  
    public void findPersonByIdReturnsPersonWithSameId() {  
        int expectedPersonId = 42;  
        var person = database.findPersonById(expectedPersonId);  
        assertEquals(expectedPersonId, person.getId());  
    }  
}
```

Durch Dependency Injection lassen sich Abhängigkeiten zu anderen Objekten ersetzen und ermöglichen so **unabhängige, isolierte Unit Tests!**

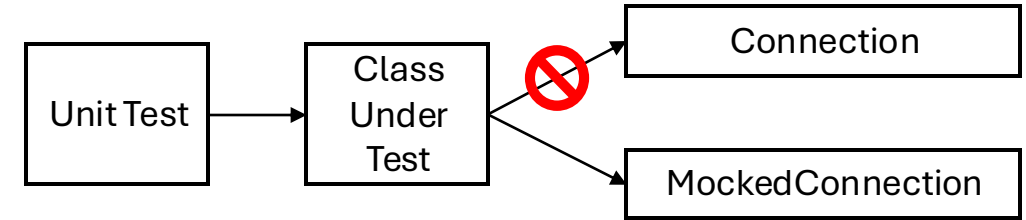
Testdouble

Ein **Testobjekt** (SUT) spricht häufig mit **anderen Objekten**, die als **Kollaborateure** oder **Abhängigkeiten** bezeichnet werden.

Um **Unit-Tests** zu **isolieren** und die **Kontrolle** über alle Aspekte des Tests zu behalten, ist es nützlich, die echten kollaborierenden Objekte durch "gefälschte" Ersatzobjekte, aka **Testdoubles**, zu ersetzen.

Testdoubles sehen aus **wie** die **Originale**, haben aber **keine Abhängigkeiten zu anderen Objekten**.

Mocks



Mocks sind Objekte bzw. Testdoubles, die das Verhalten anderer Objekte nachahmen.

Mocks sind sinnvoll, um

- Tests unabhängig von anderen Teilen des Systems zu machen.
(z. B. externe Datenbanken)
- die Ausführungszeit von Tests zu erhöhen.
(z. B. Vermeiden vieler Anfragen übers Netzwerk)
- Fehler zu simulieren, die sich nur schwer reproduzieren lassen.
(z. B. Netzwerkfehler)

Wann mocken? (nach Uncle Bob)



<https://blog.cleancoder.com/uncle-bob/2014/05/10/WhenToMock.html>

Mocks vs. Stubs (nach Martin Fowler)

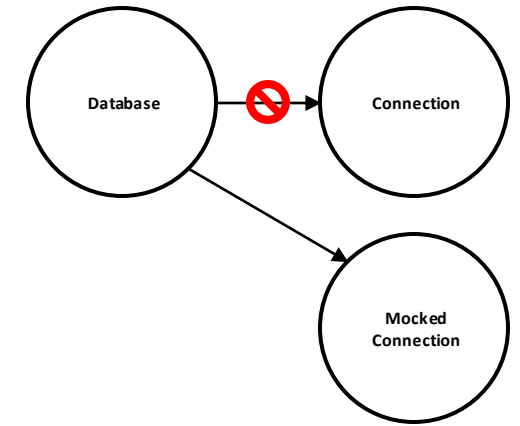


<https://martinfowler.com/articles/mocksArentStubs.html>

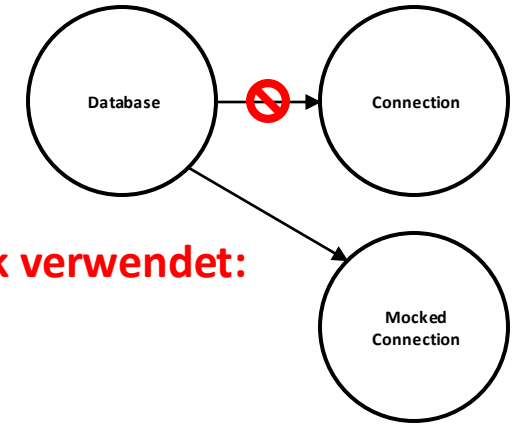
Beispiel: Mock 1/2

```
class Database {  
    private final Connection connection;  
  
    public Database(Connection connection) {  
        this.connection = connection;  
    }  
  
    public Person findPerson(int personId) {  
        return connection.query(personId);  
    }  
}
```

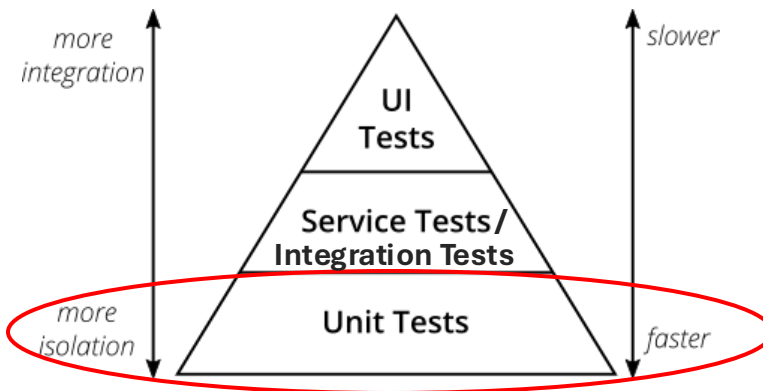
```
class DatabaseTest {  
    private static Database database;  
  
    @BeforeAll  
    static void beforeAll() {  
        database = new Database(new MockedDatabaseConnection());  
    }  
  
    @Test  
    public void findPersonByIdReturnsPersonWithSameId() {  
        int expectedPersonId = 42;  
        var person = database.findPerson(expectedPersonId);  
        assertEquals(expectedPersonId, person.getId());  
    }  
}
```



Beispiel: Mock 2/2 (Unit Test)



```
class Database {  
    private final Connection connection;  
  
    public Database(Connection connection) {  
        this.connection = connection;  
    }  
  
    public Person findPerson(int personId) {  
        return connection.query(personId);  
    }  
}
```



Unit Test, der keine reale Datenbank verwendet:

```
class DatabaseTest {  
    private static Database database;  
  
    @BeforeAll  
    static void beforeAll() {  
        database = new Database(new MockedDatabaseConnection());  
    }  
  
    @Test  
    public void findPersonByIdReturnsPersonWithSameId() {  
        int expectedPersonId = 42;  
        var person = database.findPerson(expectedPersonId);  
        assertEquals(expectedPersonId, person.getId());  
    }  
}
```

```
public class MockedDatabaseConnection implements Connection {  
    public Person query(int personId) {  
        return new Person(personId); // Dummy-Objekt für Testzwecke.  
    }  
}
```

Testdouble Arten



- **Dummy:** ein leeres Objekt, das in einem Aufruf übergeben wird (in der Regel nur, um einen Compiler zu befriedigen, wenn ein Methodenargument erforderlich ist).
- **Fake:** ein Objekt mit einer funktionalen Implementierung, aber normalerweise in einer vereinfachten Form, nur um den Test zu erfüllen (z. B. eine In-Memory-Datenbank).
- **Stub:** ein Objekt mit fest einkodiertem Verhalten, das für einen bestimmten Test (oder eine Gruppe von Tests) geeignet ist.
- **Mock:** ein Objekt mit der Fähigkeit, a) ein programmiertes erwartetes Verhalten zu haben und b) während seiner Lebensdauer auftretende Interaktionen zu verifizieren.
- **Spy:** ein Mock, das als Stellvertreter für ein bestehendes reales Objekt erstellt wird; einige Methoden können stubbed sein, während nicht-stubbed Methoden an das reale Objekt weitergeleitet werden.



Mockito

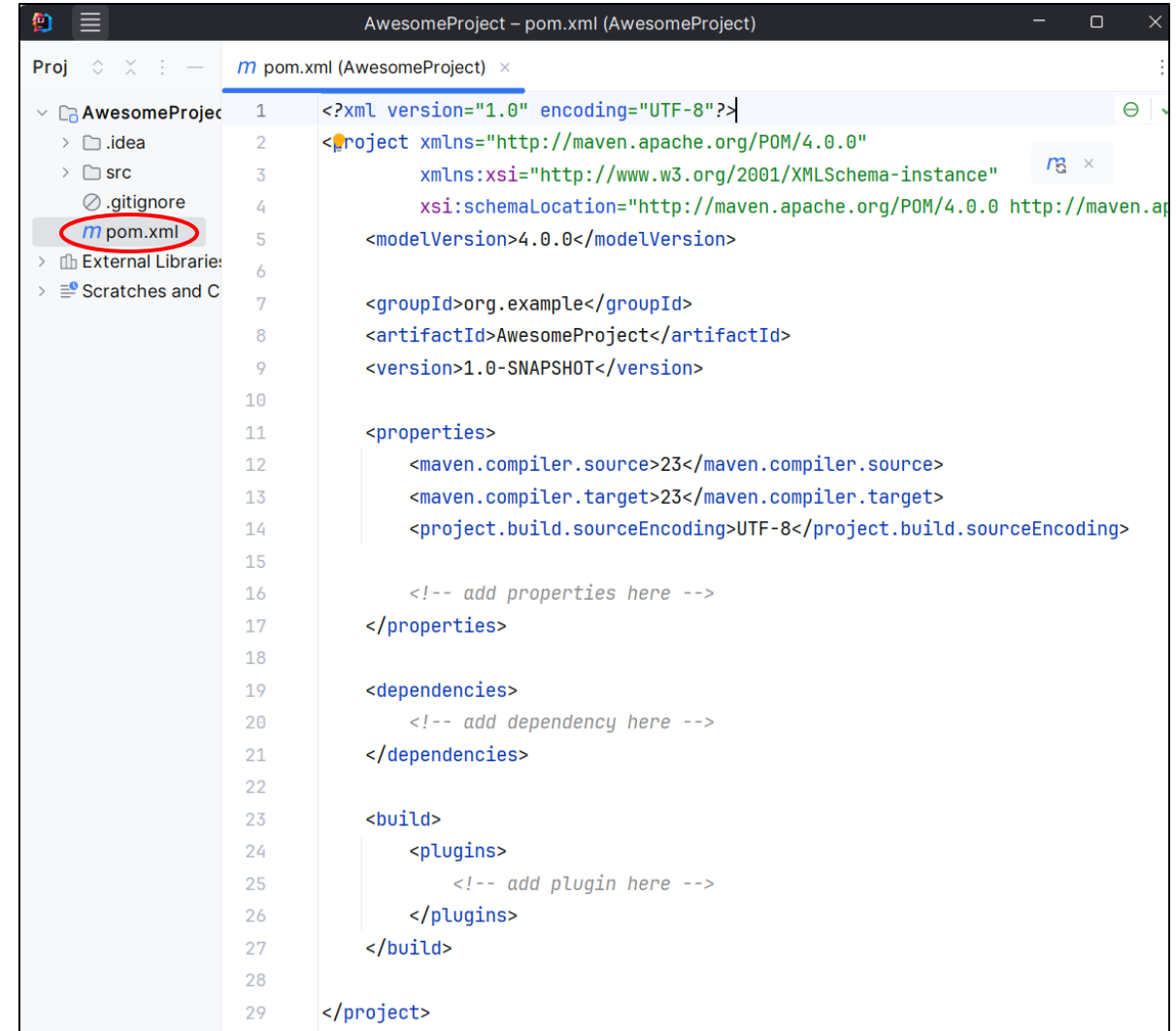
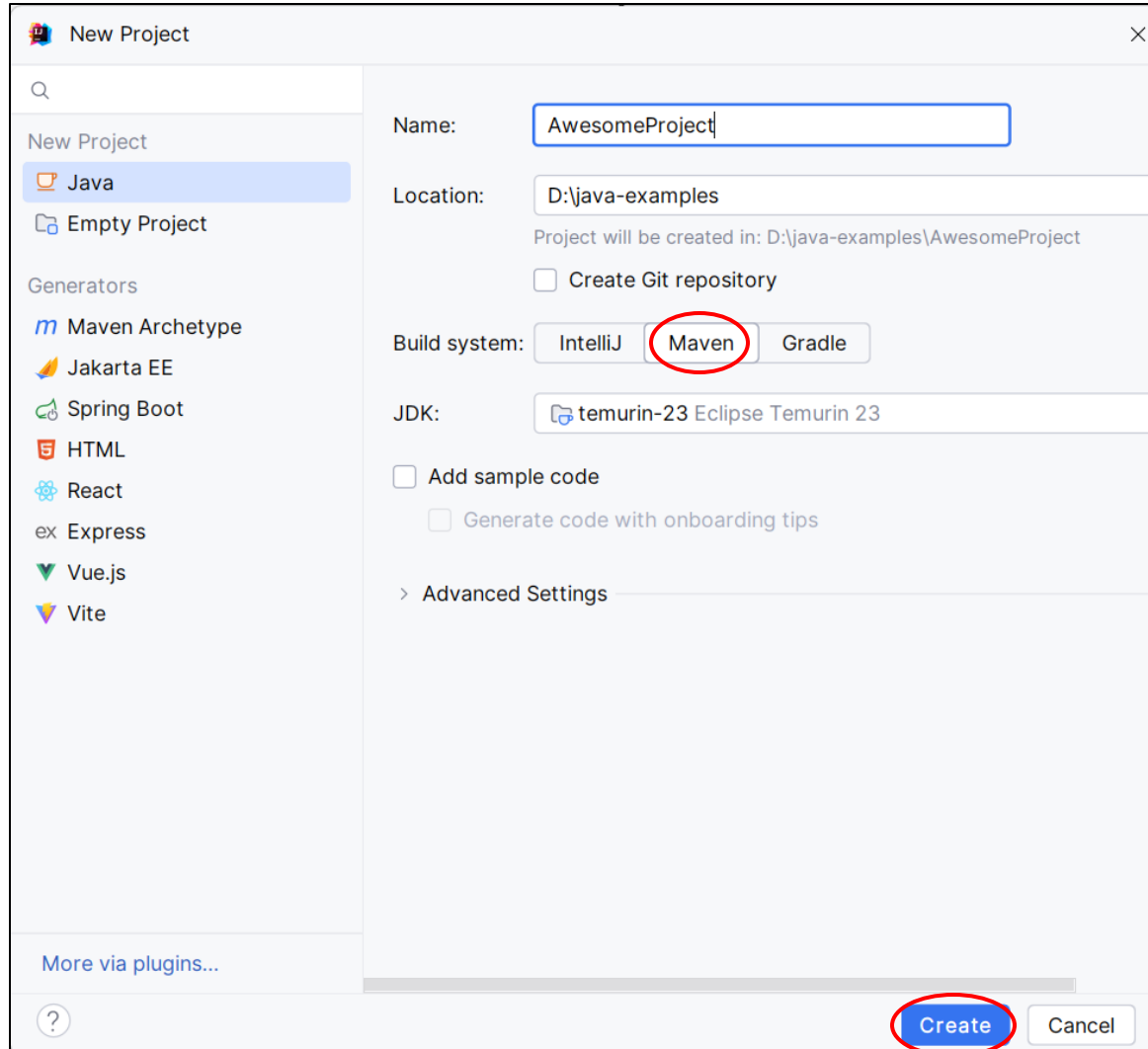


Mockito ist ein Mocking-Framework für Java, um unter anderen Testdoubles zu erstellen.

"Mit Mockito bekommt man keinen Kater, weil die Tests gut lesbar sind und Mockito saubere Fehlermeldungen produziert."

Quelle: <https://site.mockito.org/>

Mockito mit Maven 1/2



Mockito mit Maven 2/2

```
<properties>
  <!-- Java settings -->
  <maven.compiler.source>24</maven.compiler.source>
  <maven.compiler.target>24</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

  <!-- Shared version properties -->
  <mockito.version>5.17.0</mockito.version>
  <maven-surefire-plugin-version>3.5.3</maven-surefire-plugin-version>
</properties>

<dependencies>
  <!-- Mockito -->
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>${mockito.version}</version>
  </dependency>
  <!-- Needed to remove the Mockito warnings. -->
  <dependency>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>${maven-surefire-plugin-version}</version>
  </dependency>
</dependencies>
```

```
<build>
  <plugins>
    <!-- Removes inline-mock warning when using Mockito -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>${maven-surefire-plugin-version}</version>
      <configuration>
        <!-- Dynamically load mockito as an agent. -->
        <!-- A Java agent is basically a piece of software that can modify the behavior of the
        Java Virtual Machine (JVM) during runtime. In contrast to normal libraries or dependencies that
        are called directly by the application, agents "hack" into the JVM and can therefore make more
        far-reaching changes (e.g., byte code manipulations). -->
        <argLine>
          -XX:+EnableDynamicAgentLoading
          -Dnet.bytebuddy.experimental=true
          -javaagent:${user.home}/.m2/repository/org/mockito/mockito-core/${mockito.version}/mockito-core-${mockito.version}.jar
        </argLine>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Mockito: verify

Ein Mockito-Objekt **zeichnet** alle **Interaktionen auf**, d. h. alle Methodenaufrufe.

Mittels **verify** lässt sich im Unit Test prüfen, ob bestimmte **Methoden inklusive Parameter** auf dem Mock **aufgerufen wurden**.

verify prüft **wie assert-Methode**, ob die Bedingung erfüllt ist. Wenn nicht, dann schlägt der Testfall fehl.

```
import org.junit.jupiter.api.Test;
```

```
import java.util.List;
```

```
// Importiere Mockito statisch, sodass der Code klarer ist.
```

```
import static org.mockito.Mockito.mock;
```

```
import static org.mockito.Mockito.verify;
```

```
class VerifyTest {
```

```
    @Test
```

```
    void demo() {
```

```
// FYI: List würde man normalerweise nicht mocken (nur Demo).
```

```
List<String> mockedList = mock();
```

```
// Interagiere mit dem Mock-Objekt.
```

```
mockedList.add("one");
```

```
mockedList.clear();
```

```
// Verifiziere Interaktion mit Mock-Objekt (quasi wie assert).
```

```
verify(mockedList).add("one"); // wurde add mit "one" aufgerufen?
```

```
verify(mockedList).clear(); // wurde clear() aufgerufen?
```

```
    }
```

```
}
```

Mockito: verify – times()

Per Default prüft verify, ob eine Methode **genau einmal** aufgerufen wurde.

```
class ListTest {  
  
    @Test  
    void multipleCalls() {  
        List<String> mockList = mock();  
  
        mockList.add("Sara");  
        mockList.add("Sara");  
  
        verify(mockList).add("Sara");  
    }  
}
```



org.mockito.exceptions.verificatio.
TooManyActualInvocations:
list.add("Sara");
Wanted 1 time:
But was 2 times:

```
class ListTest {  
  
    @Test  
    void multipleCalls() {  
        List<String> mockList = mock();  
  
        mockList.add("Sara");  
        mockList.add("Sara");  
  
        verify(mockList, times(2)).add("Sara");  
    }  
}
```

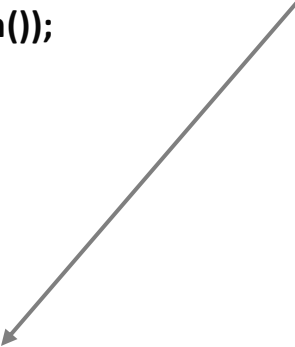


Mockito: when (Stubbing)

```
class StubbingTest {  
  
    @Test  
    void demo() {  
        // Man kann auch konkrete Klassen mocken, nicht nur Interfaces.  
        // FYI: LinkedList würde man normalerweise nicht mocken (nur Demo).  
        LinkedList<String> mockedList = mock();  
  
        // Stubbing  
        when(mockedList.get(0)).thenReturn("first");  
        when(mockedList.get(1)).thenThrow(new RuntimeException());  
  
        // Ausgabe: "first".  
        System.out.println(mockedList.get(0));  
  
        // Wirft RuntimeException.  
        System.out.println(mockedList.get(1));  
  
        // Ausgabe: "null", da .get(999) nicht gestubbed wurde (gibt stattdessen einen typabhängigen Default-Wert zurück).  
        System.out.println(mockedList.get(999));  
  
        // Obwohl es möglich ist, einen Stubbed-Aufruf zu verifizieren, ist es überflüssig.  
        verify(mockedList).get(0);  
  
        // FYI: Stubbe nur Methoden, die auch von der zu testenden Klasse aufgerufen bzw. verwendet werden.  
    }  
}
```

Mit **when** lässt sich das Verhalten eines Mocks programmieren.

Angenehm: Mockito stellt für alle Methoden, die nicht mit **when** überschrieben werden, eine Standard-Implementierung bereit, so dass man nur die Methoden in einer Klasse oder in einem Interface mocken muss, die man gerade benötigt.



Mockito: Argument Matchers

```
class ArgumentMatchers {  
  
    @Test  
    void demo() {  
        List<String> mockedList = mock();  
  
        // Stubbing mit anyInt()-Matcher  
        when(mockedList.get(anyInt())).thenReturn("element"); // wenn get mit irgendeinen int-Wert aufgerufen, dann gebe "element" zurück.  
  
        // Stubbing ist nur notwendig, wenn der Rückgabewert eine Rolle spielt (siehe .thenReturn).  
        // Ansonsten lassen sich die Methoden der gemockten Klasse auch einfach direkt aufrufen:  
        mockedList.add("Sara"); // Aufruf ohne stubbing  
  
        // Ausgabe: "element"  
        System.out.println(mockedList.get(999));  
  
        // Ausgabe: false  
        System.out.println(mockedList.contains("Sara"));  
  
        // Argument Matchers funktionieren auch mit verify.  
        verify(mockedList).get(anyInt());  
  
        verify(mockedList).add(anyString());  
    }  
}
```

Argument Matcher erlauben es, dass ein Mock auf irgendwelche Parameter reagiert. D. h., der übergebene Wert spielt keine Rolle, Hauptsache der Typ stimmt überein.

Mockito: Argument Matchers – Reihenfolge

Die Reihenfolge bei "when mit Argument Matcher" ist relevant.

```
class ListTest {  
  
    @Test  
    void multipleWhenWithMatcher() {  
        List<String> mockList = mock();  
  
        when(mockList.get(1)).thenReturn("Velvet");  
        // Folgender Matcher überschreibt vorherige Zeile.  
        when(mockList.get(anyInt())).thenReturn("Sara");  
  
        assertEquals("Velvet", mockList.get(1));  
        assertEquals("Sara", mockList.get(2));  
    }  
}
```



org.opentest4j.AssertionFailedError:
Expected :Velvet
Actual :Sara

```
class ListTest {  
  
    @Test  
    void multipleWhenWithMatcher() {  
        List<String> mockList = mock();  
  
        when(mockList.get(anyInt())).thenReturn("Sara");  
        when(mockList.get(1)).thenReturn("Velvet");  
  
        assertEquals("Velvet", mockList.get(1));  
        assertEquals("Sara", mockList.get(2));  
    }  
}
```



Mockito: Argument Matchers all or nothing

Achtung:

Sobald man einen Argument Matcher verwendet, müssen **alle** Parameter der Methode einen Argument Matcher verwenden!

*// **Falsch:** es wird eine Exception ausgelöst, weil das dritte Argument ohne einen Argument-Matcher angegeben wird.*

❌ `verify(mock).someMethod(anyInt(), anyString(), "third argument"); // falsch`

*// **Korrekt:** `eq()` ist auch ein Argument-Matcher, der für "equals" steht.*

✅ `verify(mock).someMethod(anyInt(), anyString(), eq("third argument")); // korrekt`

Mockito: Argument Matchers Kontext

Achtung:

Verwende Argument Matcher nur im Zusammenhang mit **when** oder **verify**!

```
Service peopleServiceMock = mock();
```

✓ *when*(peopleServiceMock.find(*any*())); // *korrekt, da when.*

```
peopleServiceMock.save(new Person("Sara"));
```

✗ peopleServiceMock.save(*any*()); // *falsch, da kein when oder verify.*

✓ *verify*(peopleServiceMock).save(*any*()); // *korrekt, da verify.*

// FYI: Folgende Schreibweise wäre notwendig, falls es *mehrere Überladungen* für die Methode save gäbe:

✓ *verify*(peopleServiceMock).save(*any*(Person.class)); // *korrekt, da verify.*

staticMock

Statische Methoden sind in der objektorientierten Welt problematisch, da sich diese in Unit Tests **nur schwer austauschen bzw. testen** lassen.

Beispiel:

Die statische Java-Methode **LocalDate.now()** gibt z. B. das aktuelle Datum zurück.



Will man in einem Unit Test testen, wie sich eine Applikation an Weihnachten verhält, muss man irgendwie dafür sorgen, dass `LocalDate.now()` immer das Datum von Weihnachten zurückgibt...



staticMock Beispiel



```
import java.time.LocalDate;

final class Christmas {

    static String getMessageOfTheDay() {
        LocalDate today = LocalDate.now(); // statische Methode 🗓️
        boolean isChristmas = today.getMonthValue() == 12 &&
                               today.getDayOfMonth() == 25;

        if (isChristmas) {
            return "Ho Ho Ho!";
        } else {
            return "Christmas is coming...";
        }
    }
}
```

FYI: Der Parameter `InvocationOnMock::callRealMethod` in `mockStatic` gibt an, dass die echten Methodenaufrufe verwendet werden sollen, wenn eine Methode auf dem Mock-Objekt aufgerufen wird, es sei denn, es gibt eine explizite Anweisung (z. B. `when(...).thenReturn(...)`), die das Verhalten überschreibt.

Das bedeutet, dass für alle Methoden von `LocalDate` die echte Implementierung aufgerufen wird, außer eine Methode wurde mit `when(...)` überschrieben.

```
import org.junit.jupiter.api.Test;
import org.mockito.MockedStatic;
import org.mockito.invocation.InvocationOnMock;

import java.time.LocalDate;

import static com.google.common.truth.Truth.assertThat;
import static org.mockito.Mockito.mockStatic;

class ChristmasTest {

    @Test
    void isChristmas() {
        try (MockedStatic<LocalDate> staticMock = mockStatic(LocalDate.class, InvocationOnMock::callRealMethod)) {
            // Arrange
            LocalDate christmasDate = LocalDate.of(0, 12, 25); // LocalDate.now() soll dies im Unit Test zurückgeben.
            staticMock.when(LocalDate::now).thenReturn(christmasDate); // rewrite the static method now()

            // Act
            String message = Christmas.getMessageOfTheDay();

            // Assert
            assertThat(message).isEqualTo("Ho Ho Ho!");
        }

        // Alternative Schreibweise:
        @Test
        void isNotChristmas() {
            try (MockedStatic<LocalDate> _ = mockStatic(LocalDate.class, InvocationOnMock::callRealMethod)) {
                // Arrange
                LocalDate noChristmasDate = LocalDate.of(0, 12, 1); // LocalDate.now() soll dies im Unit Test zurückgeben.
                // Die Variable staticMock wird hier nicht verwendet, muss aber trotzdem in try-with-resources zugewiesen werden.
                // Für Variablen bzw. Parameter, die nicht verwendet werden, hat sich etabliert ein _ als Name zu verwenden.
                when(LocalDate.now()).thenReturn(noChristmasDate); // rewrite the static method now()

                // Act
                String message = Christmas.getMessageOfTheDay();

                // Assert
                assertThat(message).isEqualTo("Christmas is coming...");
            }
        }
    }
}
```

staticMock Fallstricke



Achtung: Das Stubbing wird von Mockito vor allen anderen Methodenaufrufen zuerst durchgeführt. Mockito setzt deshalb voraus, dass der gesamte Stub inklusive aller Parameter wie z. B. bei `when(...).thenReturn(...)` vollständig initialisiert wurde.

```
import java.time.LocalDate;

final class Christmas {

    static String getMessageOfDay() {
        LocalDate today = LocalDate.now(); // statische Methode 🏠
        boolean isChristmas = today.getMonthValue() == 12 &&
                               today.getDayOfMonth() == 25;

        if (isChristmas) {
            return "Ho Ho Ho!";
        } else {
            return "Christmas is coming...";
        }
    }
}
```

```
import org.junit.jupiter.api.Test;
import org.mockito.MockedStatic;
import org.mockito.invocation.InvocationOnMock;

import java.time.LocalDate;

import static com.google.common.truth.Truth.assertThat;
import static org.mockito.Mockito.mockStatic;

class ChristmasTest {

    @Test
    void isChristmas() {
        try (MockedStatic<LocalDate> staticMock = mockStatic(LocalDate.class, InvocationOnMock::callRealMethod)) {
            // Arrange
            // Mockito benötigt als Parameter für thenReturn ein fertiges Objekt.
            // Wenn stattdessen direkt LocalDate.of als Parameter übergeben wird, erhält man eine Exception:
            // "Unfinished stubbing detected here"
            staticMock.when(LocalDate::now).thenReturn(LocalDate.of(0, 7, 25)); // Schlägt fehl. 🚫

            // Korrekt:
            LocalDate noChristmasDate = LocalDate.of(0, 7, 25);
            staticMock.when(LocalDate::now).thenReturn(noChristmasDate); // Funktioniert. 👍

            // Act
            String message = Christmas.getMessageOfDay();

            // Assert
            assertThat(message).isEqualTo("Ho Ho Ho!");
        }
    }
}
```

staticMock vs Dependency Injection

Statische Methoden lassen sich manchmal durch geschicktes Dependency Injection ersetzen.

```
import java.time.LocalDate;

@FunctionalInterface
interface DateProvider {
    LocalDate getCurrentDate();
}

class Christmas {

    private DateProvider dateProvider;

    // Im Unit Test kann ein spezieller dateProvider übergeben (injiziert) werden.
    Christmas(DateProvider dateProvider) {
        this.dateProvider = dateProvider;
    }

    String getMessageOfTheDay() {
        LocalDate today = dateProvider.getCurrentDate(); // verwende injiziertes Objekt.
        boolean isChristmas = today.getMonthValue() == 12 && today.getDayOfMonth() == 25;
        return isChristmas ? "Ho Ho Ho!" : "Christmas is coming...";
    }
}
```

```
import org.junit.jupiter.api.Test;
import java.time.LocalDate;
import static com.google.common.truth.Truth.assertThat;

class ChristmasTest {

    @Test
    void isChristmas() {
        // Arrange
        LocalDate christmasDate = LocalDate.of(2024, 12, 25);
        Christmas christmas = new Christmas() -> christmasDate; // Dependency Injection 

        // Act
        String actual = christmas.getMessageOfTheDay();

        // Assert
        assertThat(actual).isEqualTo("Ho Ho Ho!");
    }

    @Test
    void isNotChristmas() {
        // Arrange
        LocalDate notChristmasDate = LocalDate.of(2024, 12, 1);
        Christmas christmas = new Christmas() -> notChristmasDate; // Dependency Injection 

        // Act
        String actual = christmas.getMessageOfTheDay();

        // Assert
        assertThat(actual).isEqualTo("Christmas is coming...");
    }
}
```

Mockito: Weitere Features

- Bedingungen für Parameter festlegen (argThat und Custom Argument Matchers).
- Prüfen, wie oft eine Methode aufgerufen wurde (times, never).
- Prüfen, ob eine Methode innerhalb einer bestimmten Zeit antwortet (Timeout).
- Aufrufe an ein reales Objekt spionieren (spy).

Und vieles mehr, siehe [Doku](#).

Bedenke: Mit großer Macht kommt große Verantwortung!



Best Practices: Mocks



- Mocke keine Typen (Klassen), die du nicht besitzt (z. B. von externen Bibliotheken).
- Mocke keine Value Objects (i. d. R. immutable Klassen wie z. B. [record](#)).
- Mocke nicht alles!
 - Übertriebenes Mocking führt zu komplizierten und schwer wartbaren Tests.
 - Vermeide Mocks, die andere Mocks erstellen und zurückgeben.
 - Mocking-Frameworks, wie [Mockito](#), sind nicht unbedingt nötig.

Alternative: Dependency Injection mit Interfaces und selbstgeschriebene Mock-Klassen.

Mockito: Wann verwenden?

Die Stärke von Mockito ist es, **komplexe Klassen oder Interfaces** mocken zu können, weil durch Mockito nicht für jeden Mock eine neue Klasse mit allen Methoden implementiert werden muss.

Mockito ist insbesondere für **Input/Output (I/O)**-Klassen nützlich, wie z. B. Netzwerkkommunikation, Datenbankverbindungen oder Dateiverarbeitung, um einen **isolierten Unit Test** zu schreiben, der frei von **potenziellen I/O-Fehlern** ist, wie z. B. Verbindungsverlust, Hardwarefehler und Timeouts.

Trotzdem sollten **zusätzlich** noch **Integrationstests** ohne Mocks geschrieben werden, um das Zusammenspiel mit reale I/O-Klassen zu testen, da Mocks eventuell von falschen oder unrealistischen Annahmen ausgehen und auch nicht die reale Hardware testen.

Mockito: Links

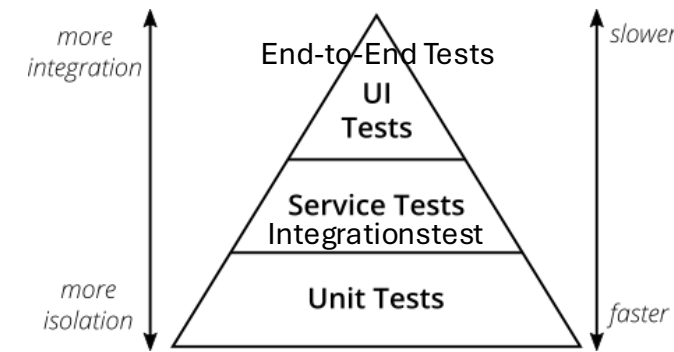
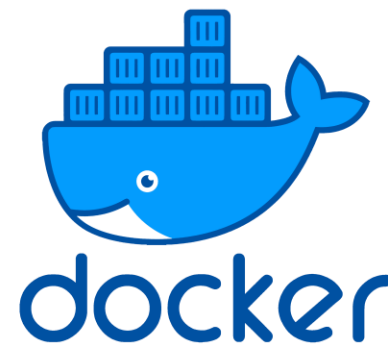
Webseite:

<https://site.mockito.org/>

Dokumentation:

<https://javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html>

Exkurs: Testcontainers



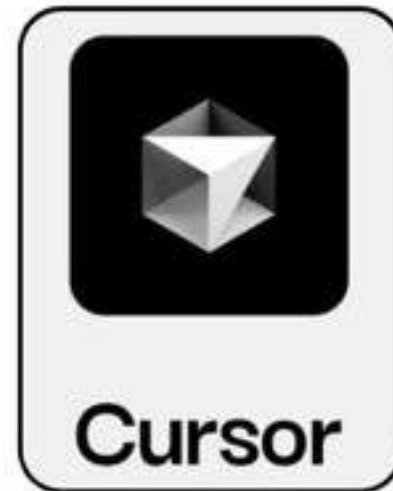
In der Softwareentwicklung werden inzwischen häufig Testcontainers eingesetzt. Diese basieren auf Docker-Container und helfen, **automatisierte Integrationstests** zu verwalten und durchzuführen.

Testcontainers starten **echte Software-Umgebungen inklusive aller Abhängigkeiten**, wie Datenbanken, Message-Broker und weitere Services.

Das Ziel ist, **Integrationstests und End-to-End-Tests realistischer** und zuverlässiger zu gestalten, ohne dass externe Dienste manuell aufgesetzt oder gemockt werden müssen.

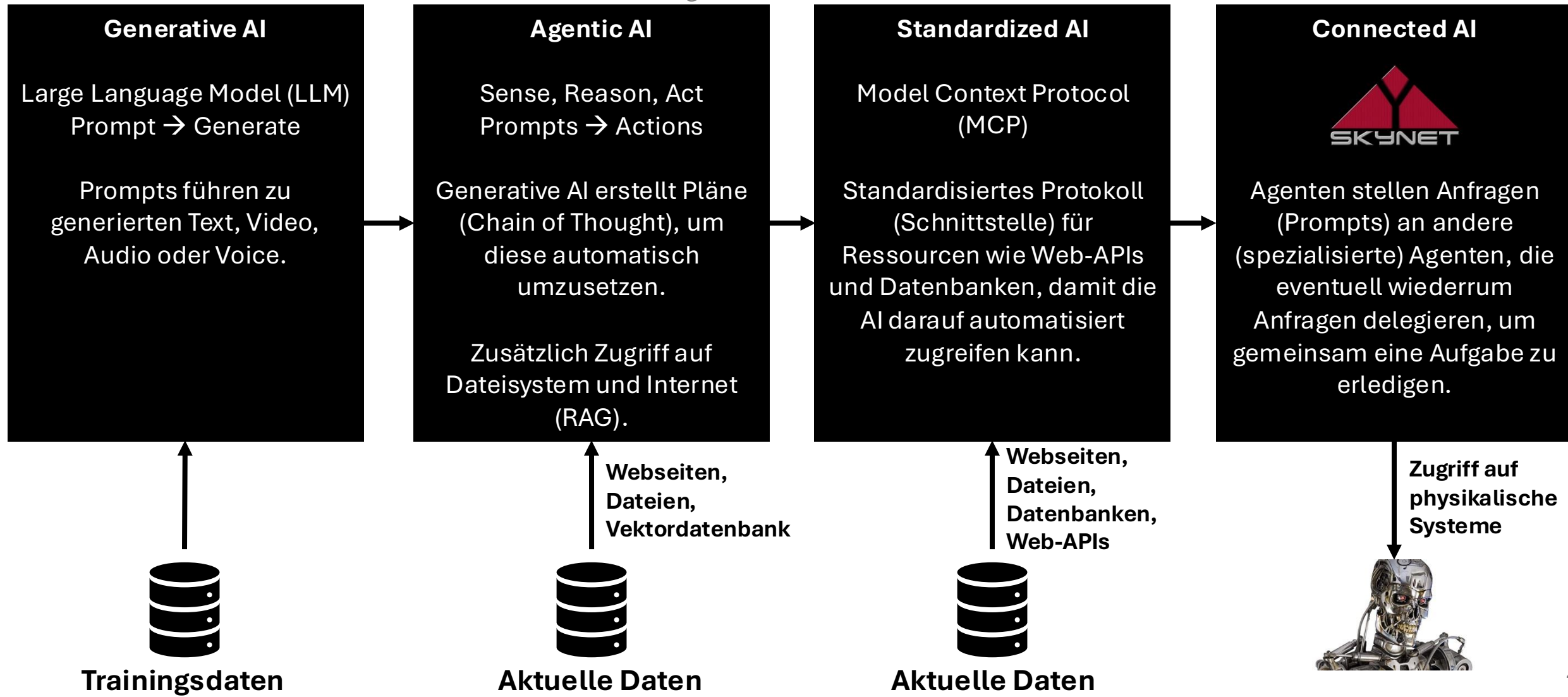
Ein großer Vorteil ist ihre **Portabilität**: da ein Testcontainer sämtliche Abhängigkeiten enthält, kann er leicht auf anderen Systemen kopiert und übertragen werden (z. B. Entwicklerrechner oder Testsystem).

Vibe Coding



Entwicklung von AI

Basis für Vibe Coding



Vibe Coding: Definition



Andrej Karpathy ✓
@karpathy



There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

Andrej Karpathy

Director of AI at Tesla.
Research Scientist and
founding member at OpenAI.

<https://karpathy.ai/>

Vibe Coding: Beispiel mit VSCode

The image shows a VS Code window with a Vue.js project. The editor is open to `App.vue`, showing a script setup with imports for `vue` and `vuedraggable`. The terminal at the bottom shows the command `npm install vuedraggable@next` being executed successfully. The chat panel on the right, powered by GitHub Copilot, provides guidance on adding drag and drop functionality, including the installation command and a suggestion to modify the `App.vue` file.

```
src > App.vue > {} script setup
1  <script setup>
2  import { ref, computed, onMounted } from 'vue'
3  import draggable from 'vuedraggable'
4
5  // State
6  const newTask = ref('')
7  const tasks = ref([])
8  const filter = ref('all') // all, active, completed
9
10 // Load tasks from localStorage on mount
11 onMounted(() => {
12   const savedTasks = localStorage.getItem('todo-tasks')
13   if (savedTasks) {
14     tasks.value = JSON.parse(savedTasks)
15   } else {
16     // Add sample tasks if no tasks exist
17     tasks.value = [
18       {
```

Keep Undo 1 of 4

OUTPUT DEBUG CONSOLE TERMINAL PORTS DBCODE

```
npm install vuedraggable@next
oliviamcvicker@Olivias-MacBook-Pro todo-app % npm install vuedraggable@next

added 1 package, and audited 51 packages in 678ms

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
oliviamcvicker@Olivias-MacBook-Pro todo-app %
```

CHAT

olguzzar

add the ability to reorder tasks by dragging

App.vue:13-41

GitHub Copilot

Used 1 reference

I'll help you add drag and drop functionality using vue-draggable. First, we need to install the required package.

npm install vuedraggable@next

Now I'll modify the App.vue file to add drag and drop functionality to the task list.

App.vue +21 -3

Working...

1 file changed

App.vue src

34 Add Context... App.vue Current file

Edit files in your workspace in agent mode

Agent Claude 3.5 Sonnet

Regeln für Vibe Coding



Daumenregeln:

1. Verwende einen weit verbreiteten Technology-Stack (Frameworks, Bibliotheken, Datenbank).
2. Übergebe ein Dokument mit Hinweisen an die AI (z. B. Guidelines, Best Practices), die sie kontinuierlich befolgt.
3. Die Aufgaben- bzw. Anforderungsbeschreibung an die AI sollte so präzise und detailliert wie möglich sein (um zu vermeiden, dass die AI kreativ wird).
4. Speichere (versioniere) funktionierenden Code mit Git.



1. **Start from a template:** Begin your project by cloning a template from GitHub or another source to provide a solid foundation. (On cursor, Start from Repo, and paste this link to build a nextjs app that is prebuilt with AI features, database and authorization <https://github.com/ansh/template-2>)
2. **Use agent mode:** Utilize Cursor's agent mode (not normal mode) to create, edit, and manage files through natural language commands.
3. **Use Perplexity:** Use perplexity to find new designs and api's from the web. Say that you are creating a nextjs project, and that you want to create ___x___ feature, and to give you instructions AND code examples.
4. **Create new chats in Composer:** Open a new Composer chat for each distinct task. Keep agent chats short.
5. **Run locally, test frequently:** Use built-in servers to run your app locally and test often to catch issues early.
6. **Iterate and refine:** Embrace rapid iteration—don't worry about perfect designs initially; improve them step by step.
7. **Utilize voice-to-text:** Use tools like *Whispr Flow* for faster input, and just vibe.
8. **Clone and fork wisely:** Use GitHub repos as starting templates to accelerate development, or to find inspiration, then customize them to fit your vision.
9. **Copy errors and paste into Composer agent:** When errors occur, copy error messages from your console and paste them into the Composer agent, and more times than not, it will be fixed. When dealing with errors, over explain the issue if it's not fixed the first time.
10. **Don't forget you can restore previous Composer chats:** Save your work frequently so you can revert to an earlier state if needed.
11. **Secure your secrets:** Always store API keys and sensitive data in environment files rather than hard-coding them.
12. **Commit often:** Push your progress to GitHub regularly to track changes and safeguard your work. Cursor can do this for you, just ask the agent.
13. **Deploy early:** Use platforms like Vercel to deploy your app early, to make sure there are no errors on deployment.
14. **Keep a record of the prompts that work best; reuse them often:** Document your most effective prompts to make future development and debugging easier.
15. **Enjoy the process—just vibe:** Embrace the creative journey of vibe coding, experiment, learn, and have fun along the way. just vibe.

Guidelines

Beispiel für Hinweise an die AI (z. B. Guidelines, Best Practices), die sie kontinuierlich befolgen soll.

BTW: Die kann man sich ebenfalls von AI generieren lassen. 😊

Best Practices for AstroJS, React, Tailwind CSS, and Nanostores

Application Structure

Project Organization

```
src/
├── components/
│   ├── ui/           # Reusable UI components
│   └── features/      # Feature-specific components
├── layouts/           # Astro layout components
├── pages/             # Astro pages (file-based routing)
├── stores/            # Nanostores state management
├── utils/             # Utility functions and helpers
├── styles/            # Global styles and Tailwind customizations
└── assets/            # Static assets like images, fonts
```

Feature Organization (for larger applications)

```
src/
├── features/
│   ├── auth/
│   │   ├── components/ # React components specific to auth
│   │   ├── stores/      # Auth-related nanostores
│   │   └── utils/       # Auth-specific utilities
│   └── products/
│       ├── components/
│       └── stores/
├── shared/             # Shared components, stores, utilities
│   ├── components/
│   └── utils/
└── pages/              # Astro pages using components from features
```

Component Creation Guidelines

When to Create a New Component

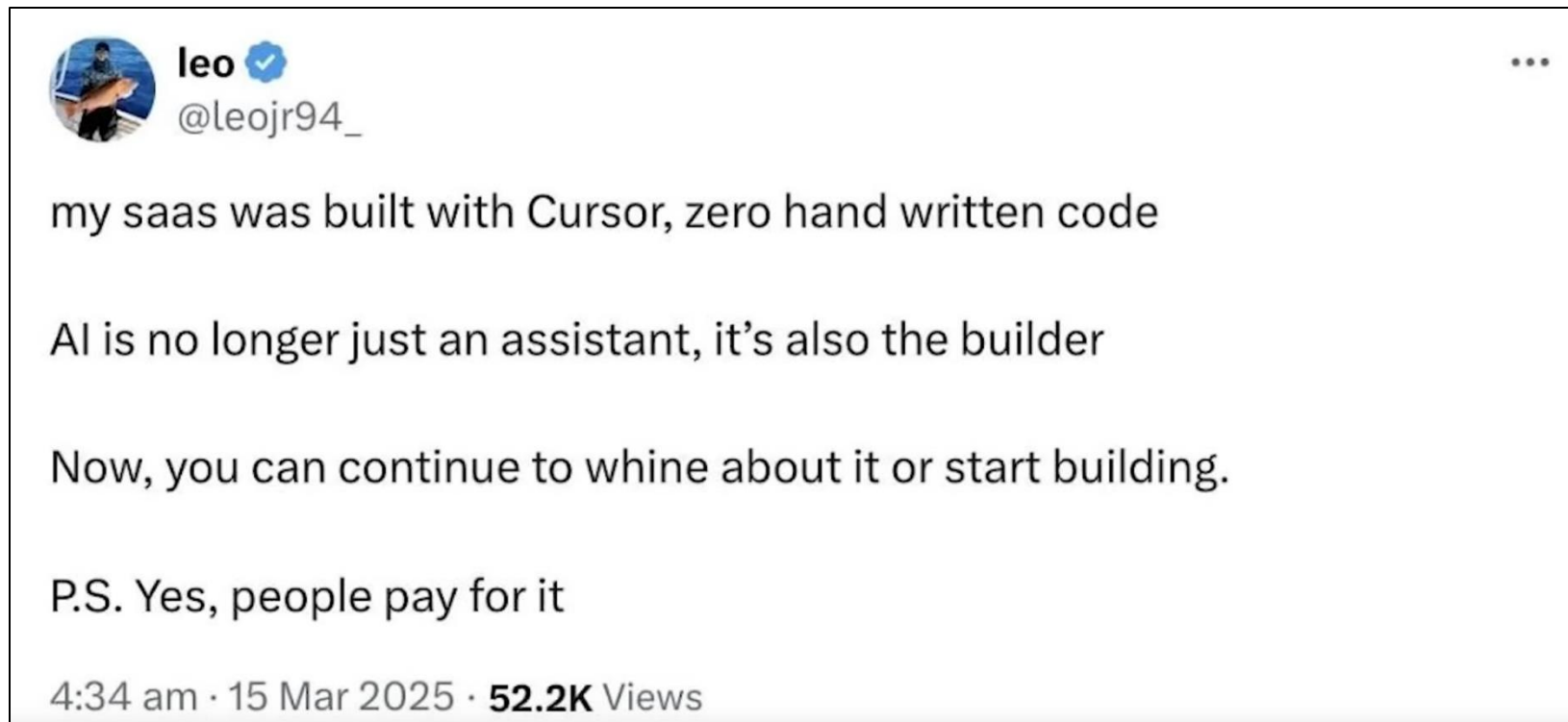
- Create a new component when:
 - UI element will be reused in multiple places
 - A section of UI exceeds 100 lines of code
 - A section has its own distinct responsibility or purpose
 - Logic for a section becomes complex (more than 3 state variables or hooks)
 - Component has a specific interaction pattern (dropdown, modal, etc.)

Anforderungen

Beispiel für ein Anforderungsdokument.

Requirement ID	Description	User Story	Expected Behavior/Outcome
FR001	Creating a New URL List	As a user, I want to be able to start a new, empty list so I can begin adding URLs.	The system should provide a clear way for the user to initiate the creation of a new list, potentially presenting an empty list view or an "add new list" button.
FR002	Adding URLs to a List	As a user, I want to be able to input or paste URLs into my list so I can compile the resources I want to share.	The system should provide an input field or mechanism for users to enter URLs. Upon submission, the URL should be added to the list.
FR003	Viewing URLs in a List	As a user, I want to be able to see all the URLs I have added to my list so I can review and manage them.	The system should display the list of added URLs, potentially with their automatically fetched titles and descriptions.
FR004	Editing URLs in a List	As a user, I want to be able to edit a URL in my list if I made a mistake or if the link has changed.	The system should provide a way for users to modify the URL of an existing item in the list.
FR005	Deleting URLs from a List	As a user, I want to be able to remove a URL from my list if it's no longer relevant or if I added it by mistake.	The system should provide a mechanism for users to delete individual URLs from their list.
FR006	Customizing the List URL	As a user, I want to be able to choose a custom URL for my list so it's easy to remember and share.	The system should provide an input field where users can specify their desired custom URL. It should also handle cases where the chosen URL is already taken.
FR007	Automatic URL Generation	As a user, if I don't want to think of a custom URL, I want the system to automatically create one for my list so I can still share it.	The system should generate a unique and shareable URL for the list if the user doesn't provide one.
FR008	Publishing a List	As a user, I want to be able to publish my list so that others can view the collection of URLs.	Upon publishing, the list should be saved and made publicly accessible at the associated URL.
FR009	Sharing a List	As a user, I want to be able to easily share the link to my list with others through various channels.	The system should provide the user with the URL of their list, potentially with options for copying it.
FR010	Accessing a Shared List	As a recipient, I want to be able to view the collection of URLs by clicking on or entering the shared link.	When a user visits a The Urlist link, they should be presented with the list of URLs.
FR011	Viewing all lists	As a user, I want to be able to see all the lists I have created so I can manage them easily.	The system should provide a dashboard or overview page where users can see all their created lists.
FR012	Deleting a List	As a user, I want to be able to delete an entire list if I no longer need it.	The system should provide a way for users to delete their entire list, including all associated URLs.

Vibe Coding: Praxisbeispiel @leojr94_ 1/2

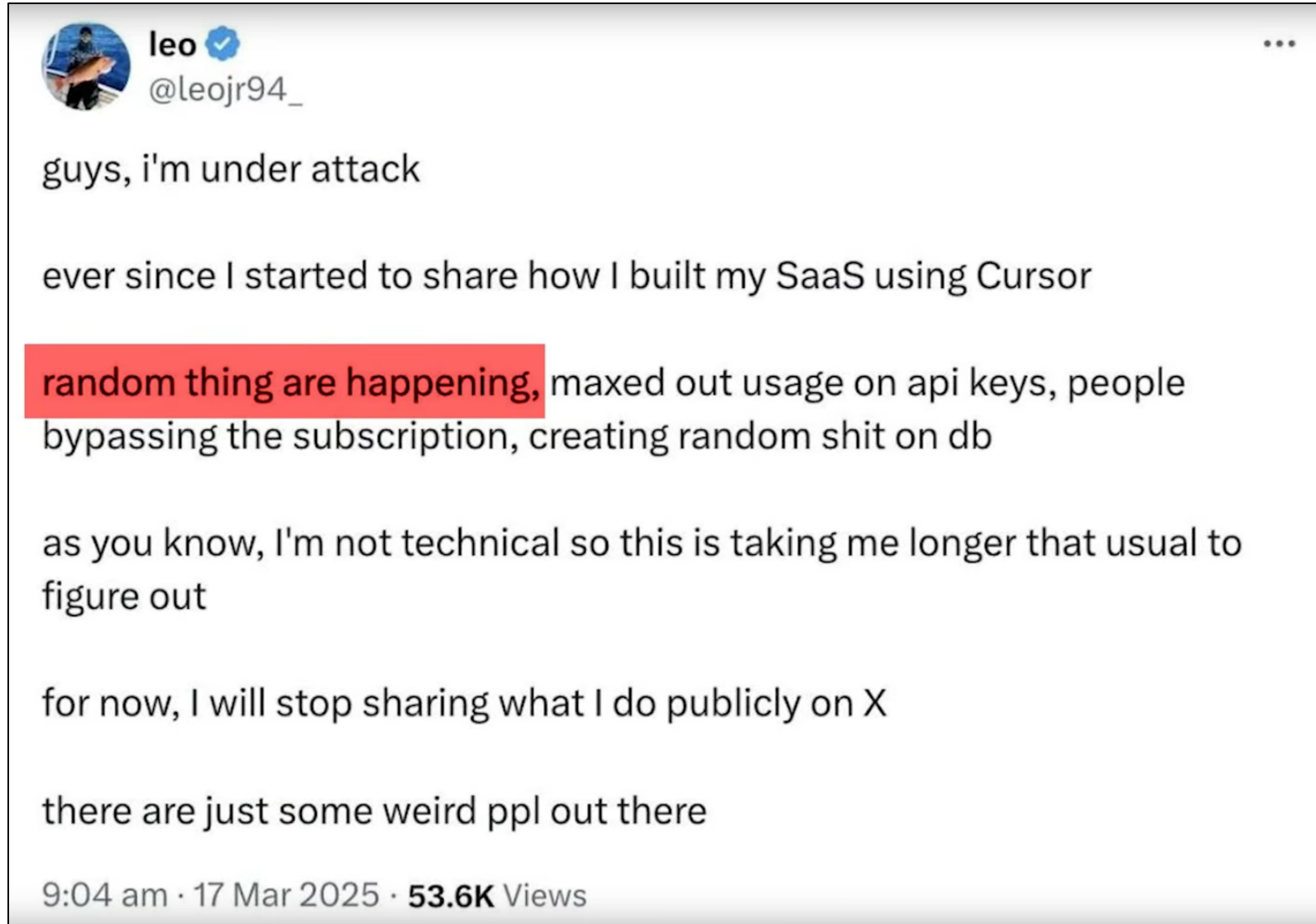


SaaS:

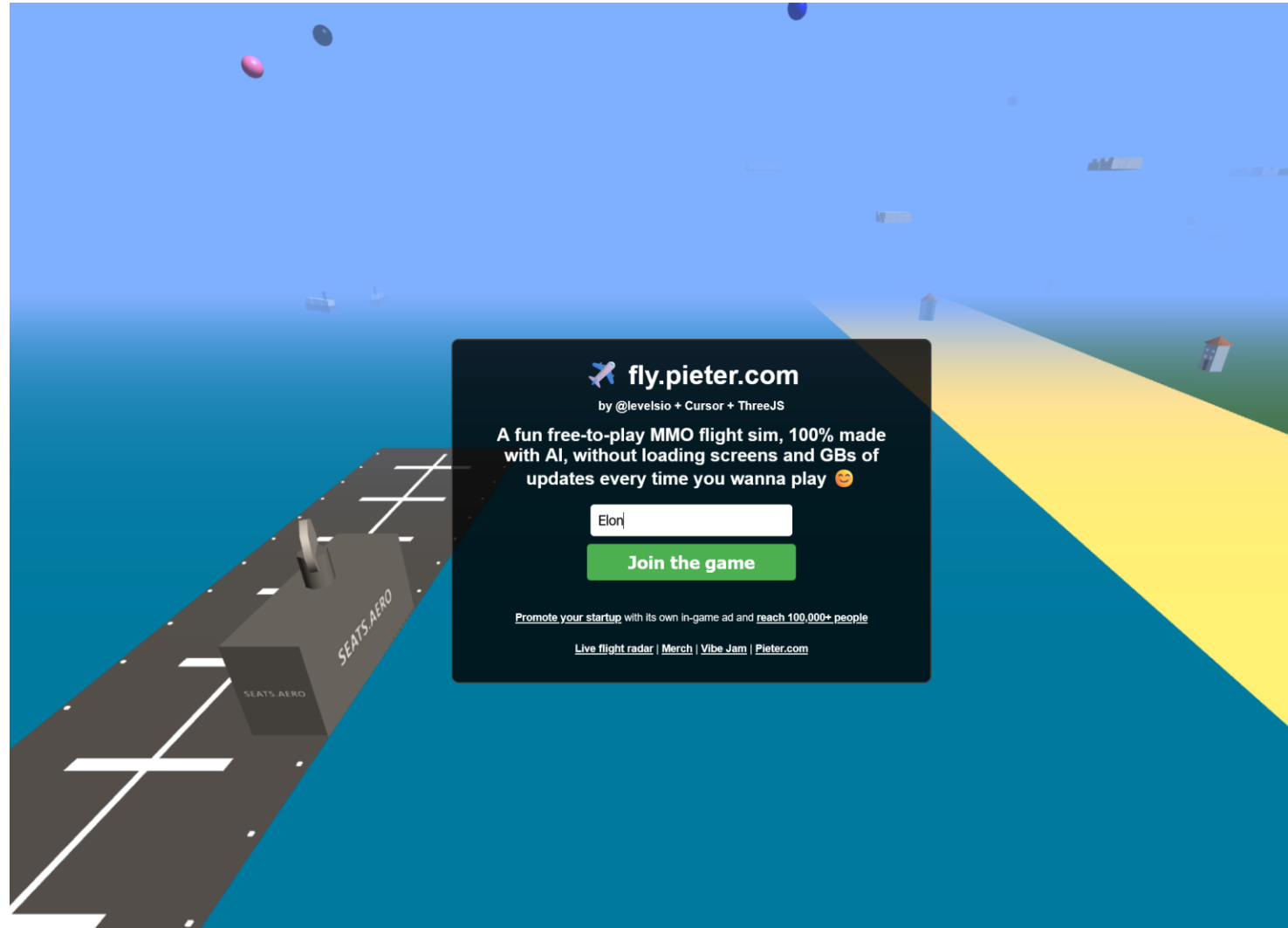
Bei Software-as-a-Service (SaaS) handelt sich um ein Cloud-basiertes Bereitstellungsmodell, bei dem Softwareanwendungen über das Internet als Dienst bereitgestellt werden – anstatt sie lokal auf dem Computer eines Nutzers zu installieren.

Beispiele: Outlook, Gmail, Jira, Slack, Trello, Github

Vibe Coding: Praxisbeispiel @leojr94_ 2/2

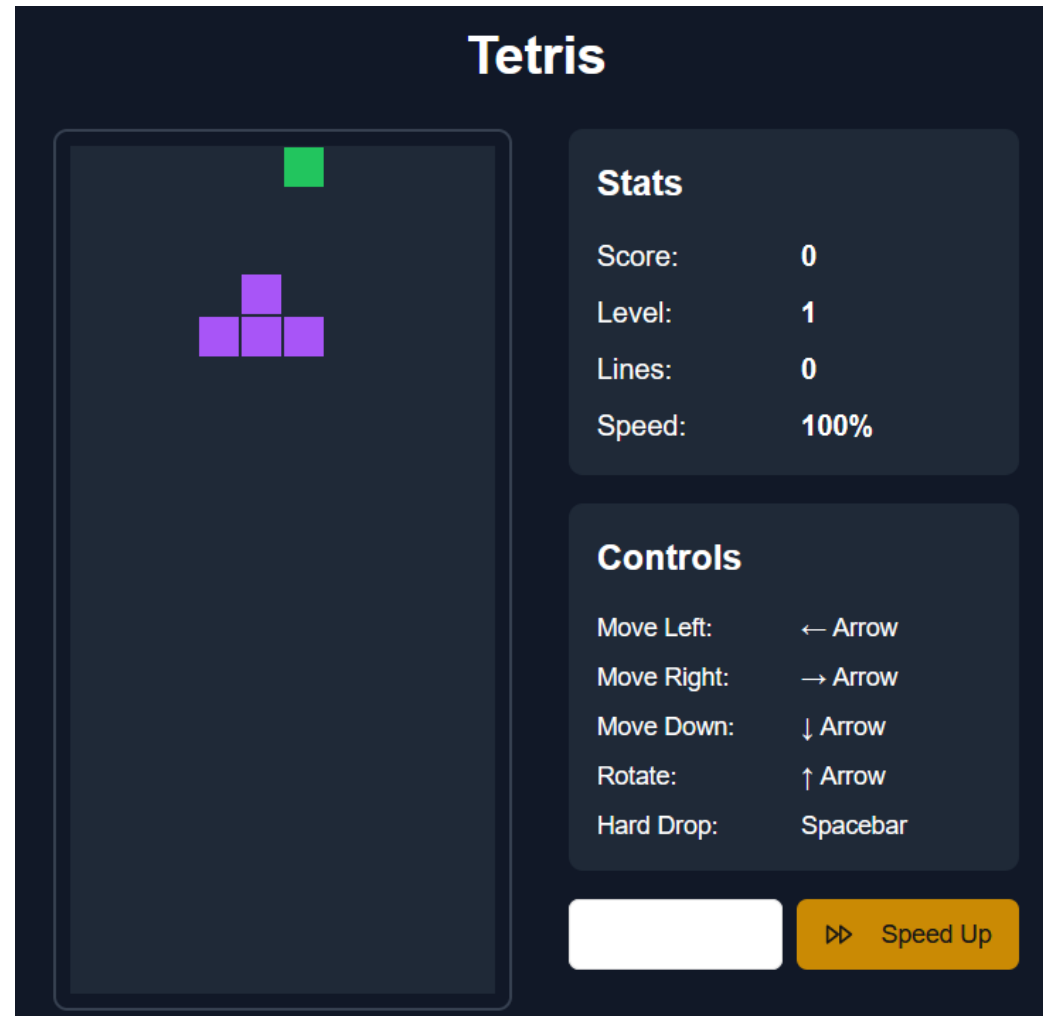


Vibe Coding: Praxisbeispiel fly.pieter



<https://fly.pieter.com/>

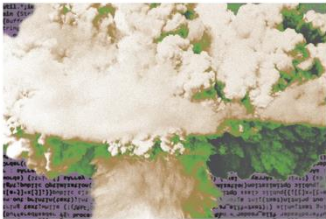
Vibe Coding: Praxisbeispiel v0-tetris



<https://v0-browser-tetris-game.vercel.app/>

Vibe Coding: Job-Angebot

Es sind bereits erste Job-Angebote als "AI Vibe Coder" aufgetaucht...



BUSINESS
Vibe Coding Is Coming for Engineering Jobs
BY WILL KNIGHT

AI Vibe Coder

Beyond Chiefs [🔗](#)

20457 Hamburg
Hybrides Arbeiten

25 € pro Stunde - Werkstudent

[Schnellbewerbung](#)

[🔖](#)

[🔗](#)

Stellenbeschreibung

Gehalt
25 € pro Stunde

Anstellungsart
Werkstudent

Arbeitsort

📍 20457 Hamburg

Leistungen

Aus der vollständigen Stellenbeschreibung

- Homeoffice-Möglichkeit

Vollständige Stellenbeschreibung

Role Overview: A Vibe Coder is a unique hybrid of Product Manager, Engineer, and UX thinker who designs experiences, questions roadmaps, and iterates until it feels right.

Core Responsibilities:

- **Building & Shipping:** Architect and launch end-to-end product features without bureaucracy
- **Design Through Code:** Create intuitive interfaces without being a formal designer
- **User-Centered Engineering:** Make technical and aesthetic decisions focused on user experience
- **Strategic Planning:** Own feedback collection and roadmap decisions without PM oversight
- **Solo Execution:** Build independently while knowing when to collaborate

Technical Requirements:

- Strong foundation in TypeScript and Python for AI development
- Experience with AI-first development tools (Cursor, bolt, n8n etc. AI-powered agents)
- Proficiency in strongly-typed languages

Quelle

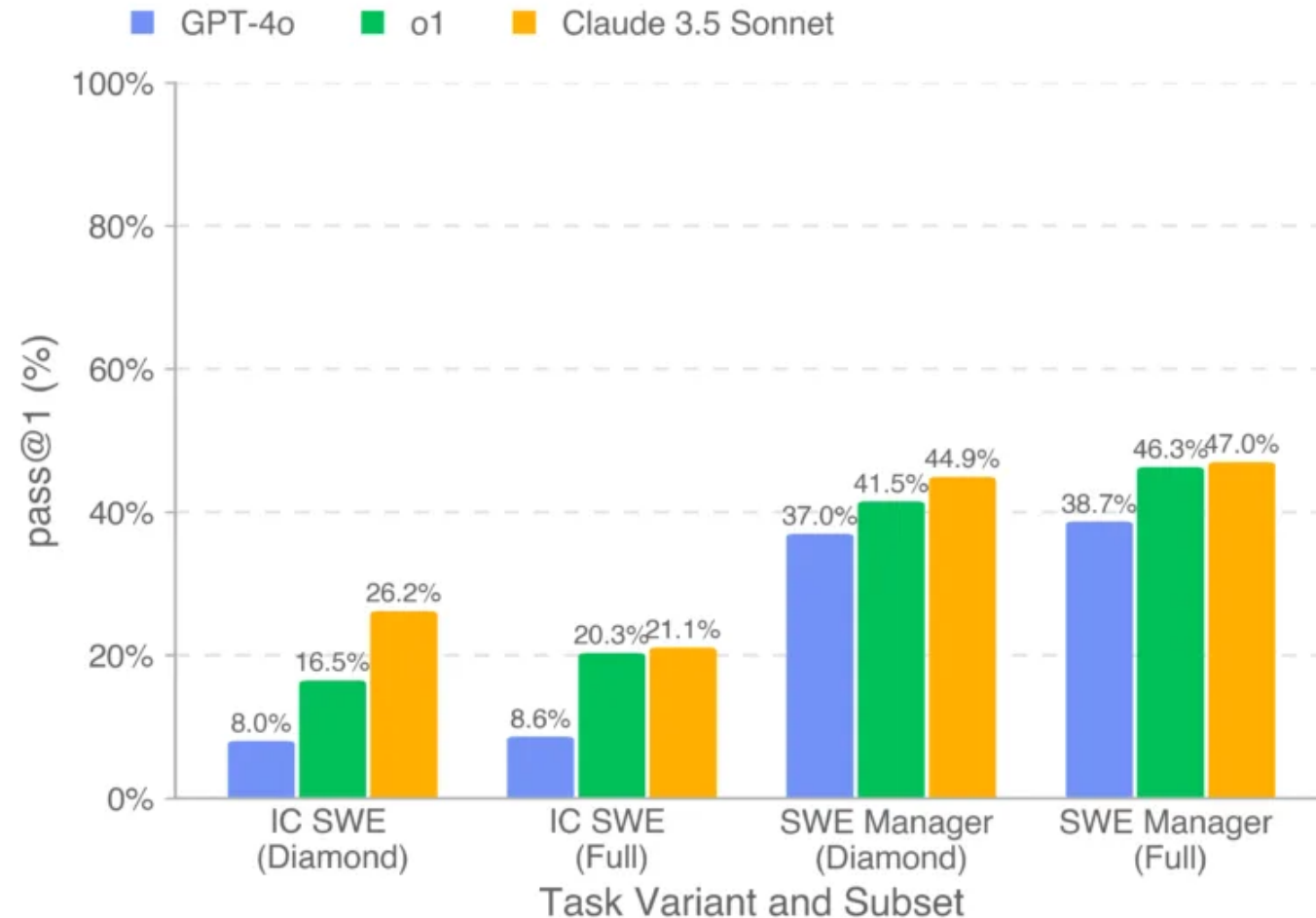
Vibe Coding: Tools (Auswahl)

- <https://www.cursor.com/>
- <https://windsurf.ai/org/de>
- <https://v0.dev/>
- <https://bolt.new/>
- <https://firebase.studio/>
- <https://github.com/openai/codex>
- <https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/overview>
- <https://www.jetbrains.com/junie/>
- <https://code.visualstudio.com/docs/copilot/chat/chat-agent-mode>
- <https://github.com/features/copilot>
- <https://superwhisper.com/> (speech to text)

Stand: AI in der Softwareentwicklung

"Ein neuer Benchmark von OpenAI zeigt, dass KI-Modelle im Praxistest zwar Potenzial zeigen, aber noch weit davon entfernt sind, komplexe Softwareprojekte eigenständig zu bewältigen."

"Eine detaillierte Analyse der Ergebnisse offenbarte eine typische Schwachstelle der KI: Die Modelle konnten zwar oft die fehlerhafte Stelle im Code lokalisieren, scheiterten aber häufig daran, die zugrundeliegende Ursache zu verstehen und eine umfassende Lösung zu entwickeln."



Quelle: <https://the-decoder.de/openai-testet-ki-als-softwareentwickler-fast-die-haelfte-des-code-budgets-geht-an-die-maschine/> (19. Februar 2025)

Model Collapse (AI-Degeneration)



"We find that indiscriminate use of **model-generated content in training causes irreversible defects in the resulting models**, in which tails of the original content distribution disappear. We refer to this effect as ‘model collapse’ and show that it can occur in LLMs"

"To sustain learning over a long period of time, we need to make sure that access to the **original data source is preserved and that further data not generated by LLMs remain available over time**. The need to **distinguish data generated by LLMs from other data** raises questions about the provenance [Herkunft] of content that is crawled from the Internet: it is unclear how content generated by LLMs can be tracked at scale."

AI in der Softwareentwicklung

Aufgrund von **Halluzinationen** und **Nicht-Determinismus** ist der Einsatz von AI in der Softwareentwicklung aktuell riskant.

Ein **möglicher Ansatz** zur Abmilderung der Risiken könnten **automatisierte Tests** darstellen, um zu prüfen, ob der generierte Code die **Anforderungen erfüllt**.



Vibe Coding ist der größte Müll

Allgemein

Es ist wirklich in aller Munde und der Gipfel des BS.

Ich hab das mal probiert, mein Setup sah so aus:

VS Code mit Cline, verbunden mit copilot claude sonnet 3.5.

1. Es ist langweilig wie sau. Schreiben, warten, schreiben, warten...
2. Du hast einen Berg voll Code vor dir und wenn irgendwas nicht funktioniert, kannst du länger debuggen als die Zeit, die man es hätte selbst schreiben können.
3. Man lernt nichts, im Gegenteil. man verdummt noch mehr, weil man nur noch LLM's vor sich her schubst und irgendwann ist man aufgeschmissen wenn man das viele Jahre so durchzieht.
4. Denkt da mal einer an Security?

Was habt ihr für Erfahrungen gemacht?

↑ 250 ↓

💬 68



↗ Share

🚩 Report

Share your thoughts

Sort by: Best ▾

🔍 Search Comments



Nullsummenspieler • 7mo ago

Der logische nächste Schritt ist Vibe Debugging und dann nur noch Management.

Damit werden wir endlich alle zu echten Tastaturaffen.



↑ 125 ↓

💬 Reply

🏆 Award

↗ Share



sogo00 • 7mo ago • Edited 7mo ago

Cursor IDE: rechts-click auf die Fehlermeldung; „fix with AI“



↑ 26 ↓

💬 Reply

🏆 Award

↗ Share

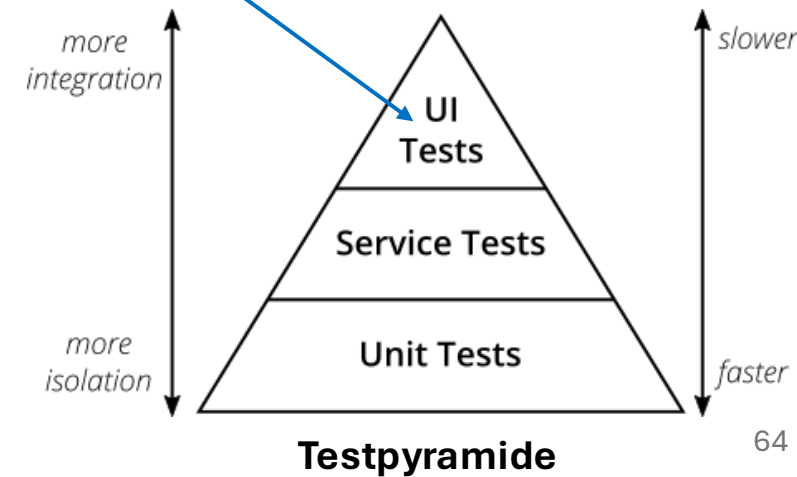


Exkurs: Akzeptanztest (Abnahmetest)



Akzeptanztest ist eine Teststufe in der Softwareentwicklung, bei der überprüft wird, **ob ein System die Anforderungen und Erwartungen des Kunden oder Nutzers erfüllt**. Es ist meist der letzte Test vor der Freigabe und Auslieferung der Software.

Akzeptanztests können **automatisiert** als **End-to-End-Test** mittels geeigneter Test-Frameworks umgesetzt werden.



Beispiel: Akzeptanztest als End-to-End-Test

Die komplette Anwendung wird hier aus Sicht des Endanwenders getestet. Die Anwendung wird dabei wie eine Blackbox behandelt – es werden Benutzereingaben simuliert und die Ausgabe überprüft, ohne zu betrachten, wie intern das Ergebnis zustande kommt.

@Test

```
void shouldAddBookToShoppingBasket() {  
    shopping.searchForBook("title: JUnit in Action");  
    shopping.selectBook("author: Catalin Tudos");  
    shopping.addSelectedItemToShoppingBasket();  
    List<String> basket = shopping.itemsInBasket();  
    assertTrue(basket.contains("item: JUnit in Action"));  
}
```

// more test cases ...

Zukunft der Softwareentwicklung mit AI (?)

Schritte:

1. Akzeptanztest (Anforderungen) als Unit Test Cases schreiben.
2. AI generiert anhand der Tests die Implementation.
3. Verifizieren der generierten Implementation mithilfe der Akzeptanztests.
4. Wiederhole Schritt 2 und 3, bis das Ergebnis passt.

Rolle verändert sich: Softwareentwickler → Produktmanager

Produktmanager sind für die Produktstrategie eines Produkts verantwortlich, **legen die Anforderungen fest** und verwalten die Veröffentlichung von Funktionen.

Natürliche vs Formale Sprachen

"In order to make machines significantly easier to use, it has been proposed (to try) to **design machines that we could instruct in our native tongues**. this would, admittedly, make the machines much more complicated, but, it was argued, by letting the machine carry a larger share of the burden, life would become easier for us. It sounds sensible provided you blame the obligation to use a formal symbolism as the source of your difficulties."

"The virtue of formal texts is that their manipulations, in order to be legitimate, need to satisfy only a few simple rules; they are, when you come to think of it, an amazingly effective tool for **ruling out all sorts of nonsense** that, when we use our **native tongues**, are **almost impossible to avoid**."

Quelle: On the foolishness of "natural language programming" – Edsger W.Dijkstra (1978)

<https://www.cs.utexas.edu/~EWD/transcriptions/EWD06xx/EWD667.html>